# DONALD B. HAVERY

**Applied AI Engineer, Forward-Deployed Focus · Agentic AI in Production · Evaluation · RAG · Tool Calling and MCP · Python and TypeScript**

Portland, OR Metro Area, Washington resident (remote or Bellevue; 25-30% travel) | dbhavery@gmail.com | github.com/dbhavery | linkedin.com/in/dbhavery | dbhavery.dev

## SUMMARY

I build agentic AI solutions that run inside a business and connect to its data. For each step of a workflow I decide what the agent handles, what conventional code handles, and what goes to a person, and I evaluate AI output before anything ships. My production customer-service agent runs on a live small-business site with Stripe checkout. It quotes prices only through tool calls against the price book, and it hands any case it cannot close to a person with the reason and the transcript. I work in Python and TypeScript with LLMs, RAG pipelines, tool calling and MCP, and I build with Claude Code daily.

## PRODUCTION SYSTEMS

**Customer-service agent in production on a revenue-taking site**

- Answers customer questions from the business's knowledge base and quotes from the price book by tool call. The model never states a price without a tool call.
- Every reply and every tool result passes the same guards before the customer sees it. A reply that contains a price with no tool call behind it is stopped and handed to a person.
- Human in the loop by design: the agent hands off on customer request, low confidence, repeated failure, frustration, prompt-injection flags, guard violation, undecided business policy or model error, and each handoff carries the transcript tail and the customer's contact when given.
- When a customer asks about a policy the business has not decided, the retrieval tool returns no answer and the case goes to a person. The rule is enforced in code, so the prompt cannot override it.
- Next.js, TypeScript, Claude API, Stripe, Vercel. Built and run solo.

**Release gate for LLM and RAG output**

Prompt assertions, retrieval precision and recall, citation grounding, comparison against a baseline. Exits non-zero on a regression, so a worse release fails CI. A mutation suite plants defects and confirms each one is caught. Python, MIT.

**Agent loop with failure handling**

Typed tool schemas, structured traces, bounded retry with backoff, a watchdog on every call, cost and step ceilings, an unsafe-action gate, and a six-class failure taxonomy. A recorded run shows a real hang cut off by the watchdog and a halt at the cost ceiling. Python, MIT.

**LLM gateway with provider failover**

OpenAI-compatible proxy with model routing, cross-provider failover behind circuit breakers, rate limiting, an exact-match response cache, token cost tracking, and an HNSW vector index written without a library. Python, MIT.

**Policy-control evidence review**

Maps control statements to supplied evidence and writes a verdict per control with the matched excerpt. Redacts secrets, keeps an append-only audit log, and leaves the decision with the reviewer. Python.

**Also shipped**

Compliance gap analysis app on Hugging Face Spaces that checks controls against SOC 2, GDPR and HIPAA and lists the gaps · weather workstation on a 3D globe for desktop and Android · CDL training site · fuel-delivery operations platform with dispatch, customer, driver, franchise and HQ roles · self-correcting text-to-CAD loop

## TECHNICAL SKILLS

| | |
|---|---|
| **Agentic AI** | Tool calling, Model Context Protocol (MCP) servers, multi-step agent loops, human-in-the-loop handoff, output guards |
| **LLMs and Prompts** | Claude, Gemini, OpenAI-compatible APIs, Qwen and Ollama local serving; prompt engineering and failure analysis |
| **Evaluation** | Fixed test sets, regression gates in CI, retrieval precision and recall, citation grounding, mutation testing |
| **RAG and Data** | RAG pipelines, hybrid retrieval (BM25 + dense), vector search indexes, ChromaDB, Postgres, SQLite, Redis |
| **APIs and Integration** | REST APIs, FastAPI, Next.js API routes, Stripe checkout, data modeling |
| **Languages and Delivery** | Python, TypeScript / JavaScript, Rust (Tauri); Docker, Vercel, GitHub Actions; Claude Code (CLAUDE.md, hooks, subagents, skills) |

## EXPERIENCE

**Applied AI Engineer, Independent** Current

- Build, deploy and maintain AI agents and applications for my businesses, from mapping the workflow to production support.
- Map each workflow before building: which steps an agent takes, which steps code takes, and where a person approves.
- Ship with automated tests and an evaluation gate, and keep a written record of each failure and its fix.

**Line-Haul Driver, Old Dominion Freight Line** Current

- Class A CDL with hazmat, tanker and triples endorsements. Company-certified driver trainer, certified through the carriers I have worked for. Daily work inside a freight operation run on dispatch and federal hours-of-service rules.

**Founder and Operator**

angrynirds, a web and AI studio for small business, live and taking payments, with the production agent above on its front desk · Trucking Tutor, a live CDL training site · a DTC grooming brand. Built and run alongside full-time work.

## EDUCATION

**Capella University**, Master of Business Administration (in progress) | **Capella University**, B.S. Computer Science (CeDiD 268P-KA6P-D3YN)

## CERTIFICATIONS AND RESEARCH

**Anthropic teaching certificate**, Teaching the AI Fluency Framework, plus **21 further Anthropic Academy certifications** across Claude, Claude Code, MCP, and enterprise deployment on Bedrock and Vertex AI

**IBM**, AI Product Manager Professional Certificate | **DeepLearning.AI**, Generative AI with LLMs

**"Reference-Relative Believability"**, independent research paper on evaluating AI-generated media, archived on Zenodo. DOI 10.5281/zenodo.21138188